

Lecture 8: Data Manipulation and Analysis (II) – Tidyverse

Harris Coding Camp – Standard Track

Summer 2024

Tidyverse vs. base R

Tidyverse	Base R	Operation
<code>select()</code>	<code>subset()</code> OR operators	extract columns
<code>filter()</code>	<code>subset()</code> OR operators	extract rows
<code>arrange()</code>	<code>sort()</code> OR <code>order()</code>	sort data
<code>rename()</code>	<code>names()</code>	rename the columns
<code>mutate()</code>	<code>\$</code> + <code><-</code>	create new columns
<code>summarize()</code>	<code>summary()</code> , <code>aggregate()</code>	summarize data

Today's Lecture

In this lecture, we will cover:

- ▶ `rename()` to rename the columns
- ▶ `mutate()` to create new columns
- ▶ `summarize()` to summarize data
- ▶ `group_by()` to make analysis across different subgroups

Renaming existing columns using `rename()`

Renaming existing columns using `rename()`

Syntax: `rename(data, new_name = old_name)`

- ▶ `data`: name of the data frame
- ▶ `new_name`: new variable name you want to keep
- ▶ `old_name`: existing variable name

```
rename(housing_data, time = year)
```

Renaming existing columns: Base R vs Tidyverse

Base R: `names(data)[names(data) == "old_name"] <- "new_name"`

Change name of year to time

`names(housing_data)[names(housing_data) == "year"] <- "time"`

Tidyverse: `rename(data, new_name = old_name)`

Change name of year to time

`rename(housing_data, time = year)`

Creating new columns with `mutate()`

.

Creating new columns with `mutate()`

storm	wind	pressure	date		storm	wind	pressure	date	ratio	inverse
Alberto	110	1007	2000-08-12	→	Alberto	110	1007	2000-08-12	9.15	0.11
Alex	45	1009	1998-07-30		Alex	45	1009	1998-07-30	22.42	0.04
Allison	65	1005	1995-06-04		Allison	65	1005	1995-06-04	15.46	0.06
Ana	40	1013	1997-07-01		Ana	40	1013	1997-07-01	25.32	0.04
Arlene	50	1010	1999-06-13		Arlene	50	1010	1999-06-13	20.20	0.05
Arthur	45	1010	1996-06-21		Arthur	45	1010	1996-06-21	22.44	0.04

We may want to create new columns such as

- ▶ $\text{ratio} = \text{pressure} / \text{wind}$
- ▶ $\text{inverse} = 1 / \text{ratio}$

Creating new columns with `mutate()`

Syntax: `mutate(data, new_var = <def>, ...)`

- ▶ `data`: name of the data frame
- ▶ `new_var`: name of a new column
- ▶ `<def>`: how the new column is defined

```
# create new column ratio = volume / listings  
mutate(housing_data, ratio = volume / listings)  
# or use pipe operator  
housing_data %>%  
  mutate(ratio = volume / listings)
```

Creating new columns with mutate()

We can create **multiple columns** at the same time and even use information from a newly created column as input.

```
# create new columns 'ratio' and 'inverse'  
housing_data %>%  
  mutate(ratio = volume / listings,  
         inverse = 1 / ratio)
```

Creating new columns with mutate()

...and keep specific columns & drop the others when needed:

```
housing_data %>%  
  mutate(ratio = volume / listings,  
         inverse = 1 / ratio) %>%  
  select(city, year, month, ratio, inverse)
```

New variables are not automatically retained unless we **assign** it to a data frame (existing or new).

```
housing_data_new <-  
  housing_data %>%  
    mutate(ratio = volume / listings,  
           inverse = 1 / ratio) %>%  
    select(city, year, month, ratio, inverse)
```

Creating new columns: Base R vs Tidyverse

Base R: data\$new_var <- <def>

```
housing_data$ratio <- housing_data$volume / housing_data$listings  
housing_data$inverse <- 1 / housing_data$ratio
```

Tidyverse: mutate(data, new_var = <def>, ...)

```
housing_data %>%  
  mutate(ratio = volume / listings,  
         inverse = 1 / ratio)
```

Creating new columns with mutate() and ifelse()

If we want to create a new column with two levels, use ifelse().

Syntax: `mutate(new_var = ifelse(<condition>, x, y))`

- ▶ `new_var`: new column to be created
- ▶ If `<condition>` is met, assign value `x` to `new_var`
- ▶ If `<condition>` is not met, assign value `y` to `new_var`

```
# create new columns 'ratio' and 'index'
housing_data %>%
  mutate(ratio = volume / listings,
         index = ifelse(ratio >= 10000, "High", "Low"))
```

Creating new columns with `mutate()` and `case_when()`

If we want more than two levels, we can use **nested** `ifelse()`, just like what we did with Base R. However, it is usually easier to use `case_when()` as it works more intuitively.

Syntax:

```
data %>%  
  mutate(new_var = case_when(  
    <condition_1> ~ x,  
    <condition_2> ~ y,  
    ...  
    <condition_final> ~ z  
  ))
```

- ▶ The arguments are evaluated in order, so only the rows where the first condition is not true will continue to be evaluated for the next condition.

Creating new columns with `mutate()` and `case_when()`

Say if we want to create a new variable called `index` with three levels (High, Medium, Low) based on the value of `ratio`:

```
housing_data %>%  
  mutate(ratio = volume / listings,  
         index = case_when(  
           ratio >= 10000 ~ "High",  
           ratio < 10000 & ratio >= 8000 ~ "Medium",  
           ratio < 8000 ~ "Low"  
         ))
```

Try it yourself – Use tidyverse

Instead of using Base R, try to use tidyverse to redo the questions:

1. Using `housing_data`, select the following variables (`city`, `year`, `month`, `sales`, `volume`) and create a new column called `mean_price`, defined as a ratio of `volume` to `sales`.
 2. Using the code from part 1, keep rows with `mean_price` at least \$50,000.
 3. Create a variable called `level` that identifies whether `mean_price` is too high (i.e. `level` equals 1 if `mean_price` is higher than \$150,000; 0.5 if `mean_price` is between \$100,000 and \$150,000; 0 otherwise).
- Hint: create `level` using `case_when()`.

Summarize data with `summarize()`

Summarize data with `summarize()`

city	particle size	amount ($\mu\text{g}/\text{m}^3$)
New York	large	23
New York	small	14
London	large	22
London	small	16
Beijing	large	121
Beijing	small	56



median
22.5

- ▶ In this case, we would like to find the mean of `amount` column among all observations.
- ▶ We may also want to find other important statistics, such as median, variance, standard deviation, etc.

Summarize data with `summarize()`

Calculate total volume of sales in Texas from 2014.

- ▶ we use `sum()`, a function within `summarize()`
- ▶ create a new column called `total_volume` which sums up all volume
- ▶ ignore missing values before doing any calculation by `summarize()`

```
housing_data %>%  
  filter(year == 2014) %>%  
  summarize(total_volume = sum(volume, na.rm = TRUE))
```

```
#> # A tibble: 1 x 1  
#>   total_volume  
#>       <dbl>  
#> 1 84760948831
```

Summarize data with summarize()

Find the max, min, mean and median of sales in Texas's three cities.

► Houston, Dallas, San Antonio

```
# Don't forget to exclude the NAs
housing_data %>%
  filter(city %in% c("Houston", "Dallas", "San Antonio")) %>%
  summarize(max_sales = max(sales, na.rm = TRUE),
            min_sales = min(sales, na.rm = TRUE),
            mean_sales = mean(sales, na.rm = TRUE),
            median_sales = median(sales, na.rm = TRUE))
```

```
#> # A tibble: 1 x 4
#>   max_sales min_sales mean_sales median_sales
#>   <dbl>     <dbl>     <dbl>         <dbl>
#> 1      8945       820      3890.         3996
```

Summarize data with `summarize()`

Find the variance and standard deviation of sales in Texas's three cities.

► Houston, Dallas, San Antonio

```
# Don't forget to exclude the NAs
housing_data %>%
  filter(city %in% c("Houston", "Dallas", "San Antonio")) %>%
  summarize(var_sales = var(sales, na.rm = TRUE),
            sd_sales = sd(sales, na.rm = TRUE))
```

```
#> # A tibble: 1 x 2
#>   var_sales sd_sales
#>   <dbl>    <dbl>
#> 1  3639848.    1908.
```

Summarize data: Base R vs Tidyverse

Base R:

```
max(housing_data$sales, na.rm = TRUE)
min(housing_data$sales, na.rm = TRUE)
median(housing_data$sales, na.rm = TRUE)
mean(housing_data$sales, na.rm = TRUE)
var(housing_data$sales, na.rm = TRUE)
sd(housing_data$sales, na.rm = TRUE)
```

Tidyverse:

```
housing_data %>%
  summarize(max_sales = max(sales, na.rm = TRUE),
            min_sales = min(sales, na.rm = TRUE),
            median_sales = median(sales, na.rm = TRUE),
            mean_sales = mean(sales, na.rm = TRUE),
            var_sales = var(sales, na.rm = TRUE),
            sd_sales = sd(sales, na.rm = TRUE))
```

Data manipulation with dplyr

The dplyr cheatsheet shows the functions which work with pipes and tidy data:

Data transformation with dplyr: : CHEAT SHEET



dplyr functions work with pipes and expect tidy data. In tidy data:



Each variable is in its own column



Each observation, or case, is in its own row



x %>% f(y) becomes f(x, y)

Summarise Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).

summary function



summarise(data, ...)
Compute table of summaries.
summarise(mtcars, avg = mean(mpg))



count(data, ..., wt = NULL, sort = FALSE, name = NULL) Count number of rows in each group defined by the variables in ... Also **tally()**.
count(mtcars, cyl)

Group Cases

Use **group_by(data, ..., add = FALSE, drop = TRUE)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.



mtcars %>%
group_by(cyl) %>%
summarise(avg = mean(mpg))

Use **rowwise(data, ...)** to group data into individual rows. dplyr functions will compute results for each row. Also apply functions to list-columns. See tidy cheat sheet for list-column workflow.



starwars %>%
rowwise() %>%

Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.



filter(data, ..., preserve = FALSE) Extract rows that meet logical criteria.
filter(mtcars, mpg > 20)



distinct(data, ..., keep_all = FALSE) Remove rows with duplicate values.
distinct(mtcars, gear)



slice(data, ..., preserve = FALSE) Select rows by position.
slice(mtcars, 10:15)



slice_sample(data, ..., n, prop, weight, by = NULL, replace = FALSE) Randomly select rows. Use n to select a number of rows and prop to select a fraction of rows.
slice_sample(mtcars, n = 5, replace = TRUE)



slice_min(data, order_by, ..., n, prop, with_ties = TRUE) and **slice_max()** Select rows with the lowest and highest values.
slice_min(mtcars, mpg, prop = 0.25)



slice_head(data, ..., n, prop) and **slice_tail()** Select the first or last rows.
slice_head(mtcars, n = 5)

Logical and boolean operators to use with filter()

`==` `<` `<=` `is.na()` `%in%` `|` `xor()`
`!=` `>` `>=` `is.na()` `!` `&`

See ?base:Logic and ?Comparison for help.

ARRANGE CASES



arrange(data, ..., by_group = FALSE) Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
arrange(mtcars, mpg)
arrange(mtcars, desc(mpg))

Manipulate Variables

EXTRACT VARIABLES

Column functions return a set of columns as a new vector or table.



pull(data, var = -1, name = NULL, ...) Extract column values as a vector, by name or index.
pull(mtcars, wt)



select(data, ...) Extract columns as a table.
select(mtcars, mpg, wt)



relocate(data, ..., before = NULL, after = NULL) Move columns to new position.
relocate(mtcars, mpg, cyl, after = last_col())

Use these helpers with **select()** and **across()**
e.g. **select(mtcars, mpg:cyl)**

contains(match) **num_range(prefix, range)** ; e.g. **mpg:cyl**
ends_with(match) **all_of(x)/any_of(x, vars)** ; e.g. **gear**
starts_with(match) **matches(match)** **everything()**

MANIPULATE MULTIPLE VARIABLES AT ONCE



across(cols, funs, ..., names = NULL) Summarise or mutate multiple columns in the same way.
summarise(mtcars, across(everything(), mean))



c_across(cols) Compute across columns in row-wise data.
transmute(rowwise(UKgas), total = sum(c_across(1:2)))

MAKE NEW VARIABLES

Apply **vectorized functions** to columns. Vectorized functions take vectors as input and return vectors of the same length as output (see back).

vectorized function



mutate(data, ..., keep = "all", before = NULL, after = NULL) Compute new column(s). Also **add_column()**, **add_count()**, and **add_tally()**.
mutate(mtcars, gpm = 1 / mpg)

Try it yourself – Use Tidyverse

Instead of using Base R, try to use **Tidyverse** to do the following tasks

Use `housing_data` and `mean_price` to complete the tasks:

1. Find the max and min of `mean_price` on the entire dataset.
2. Find the mean, median, variance and standard deviation of `mean_price` on the entire dataset.
3. Create a new variable called `new_mean_price`, which is defined as $10 * \text{mean_price} + 5$.
 - ▶ Find the max and min of `new_mean_price` on the entire dataset. Compare the your findings with part 1.
 - ▶ Find the mean, median, variance and standard deviation of `new_mean_price` on the entire dataset. Compare the your findings with part 2.
4. Redo part 2 for each city, each year. (Hint: use `group_by()`.)

```
housing_data %>%  
  group_by(city, year) %>%  
  summarize(...)
```


Analyzing data across different subgroups using
`group_by()`

Grouping data with `group_by()`

Often, we want to repeat the same analysis across different **subgroups**. We can automate that with `group_by()`. We will:

- ▶ summarize by group with `group_by()` + `summarize()`
- ▶ create new columns with `group_by()` + `mutate()` (later)
- ▶ `filter()` data with group specific matching criteria (later)

Grouping data with group_by()

Recall that count() function counts number of observations by group:

```
housing_data %>%  
  group_by(city) %>%  
  count()
```

```
#> # A tibble: 46 x 2  
#> # Groups:   city [46]  
#>   city          n  
#>   <chr>      <int>  
#> 1 Abilene      187  
#> 2 Amarillo     187  
#> 3 Arlington    187  
#> 4 Austin       187  
#> 5 Bay Area     187  
#> 6 Beaumont     187  
#> 7 Brazoria County 187  
#> 8 Brownsville  187  
#> 9 Bryan-College Station 187  
#> 10 Collin County 187  
#> # i 36 more rows
```

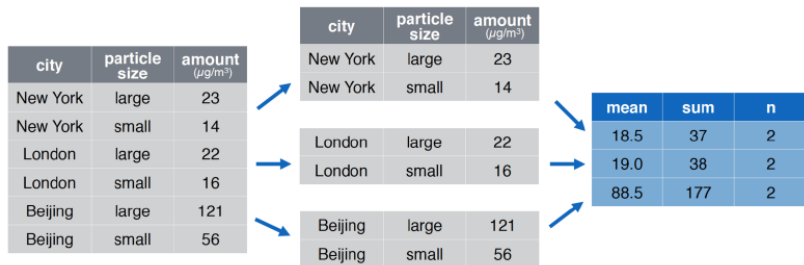
Grouping data with group_by()

Certainly, we can group by **multiple variables**. The code shows which groupings have been applied and the total number of groups.

```
housing_data %>%  
  group_by(year, city) %>%  
  count()
```

```
#> # A tibble: 736 x 3  
#> # Groups:   year, city [736]  
#>   year city          n  
#>   <dbl> <chr>      <int>  
#> 1  2000 Abilene        12  
#> 2  2000 Amarillo       12  
#> 3  2000 Arlington      12  
#> 4  2000 Austin         12  
#> 5  2000 Bay Area       12  
#> 6  2000 Beaumont       12  
#> 7  2000 Brazoria County 12  
#> 8  2000 Brownsville    12  
#> 9  2000 Bryan-College Station 12  
#> 10 2000 Collin County   12  
#> # i 726 more rows
```

Grouped summary with `group_by()` + `summarize()`



In this example, we want to calculate the mean and sum of `amount` and number of observations by `city`.

- ▶ We need to repeat the analysis across different **subgroups** (i.e. cities)
- ▶ Using `group_by()` + `summarize()` makes it a lot easier!

Grouped summary with `group_by()` + `summarize()`

Let's see the upgrouped summary statistics first:

```
housing_data %>%  
  summarize(total_sales = sum(sales, na.rm = TRUE),  
            total_volume = sum(volume, na.rm = TRUE),  
            mean_house_price = total_volume / total_sales)
```

```
#> # A tibble: 1 x 3  
#>   total_sales total_volume mean_house_price  
#>   <dbl>         <dbl>         <dbl>  
#> 1      4415202 858502159353      194442.
```

Grouped summary with `group_by()` + `summarize()`

Use case: You want summary statistics for certain subsets (each year):

- ▶ `summarize()` collapses a data frame to a single row for each group

```
annual_housing_prices <-  
  housing_data %>%  
    group_by(year) %>%  
      summarize(total_sales = sum(sales, na.rm = TRUE),  
                total_volume = sum(volume, na.rm = TRUE),  
                mean_house_price = total_volume / total_sales)  
  
head(annual_housing_prices)
```

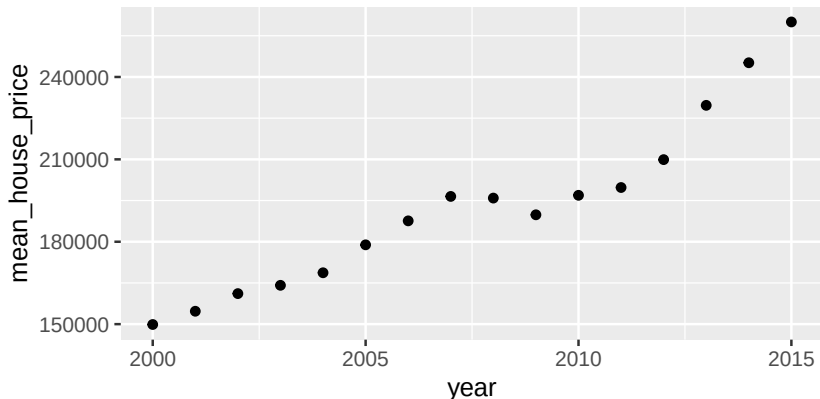
```
#> # A tibble: 6 x 4  
#>   year total_sales total_volume mean_house_price  
#>   <dbl>      <dbl>      <dbl>          <dbl>  
#> 1  2000      222483  33342410971      149865.  
#> 2  2001      231453  35804815138      154696.  
#> 3  2002      234600  37798888462      161121.  
#> 4  2003      253909  41674204834      164130.  
#> 5  2004      283999  47913188880      168709.  
#> 6  2005      316046  56534755111      178881.
```

How have Texas housing prices changed over time?

We will cover data visualization soon!

```
annual_housing_prices %>%
```

```
  ggplot(aes(x = year, y = mean_house_price)) +  
    geom_point()
```



Grouped summary with `group_by()` + `summarize()`

Use case: You want summary statistics for certain subsets (each year, each city) of the data.

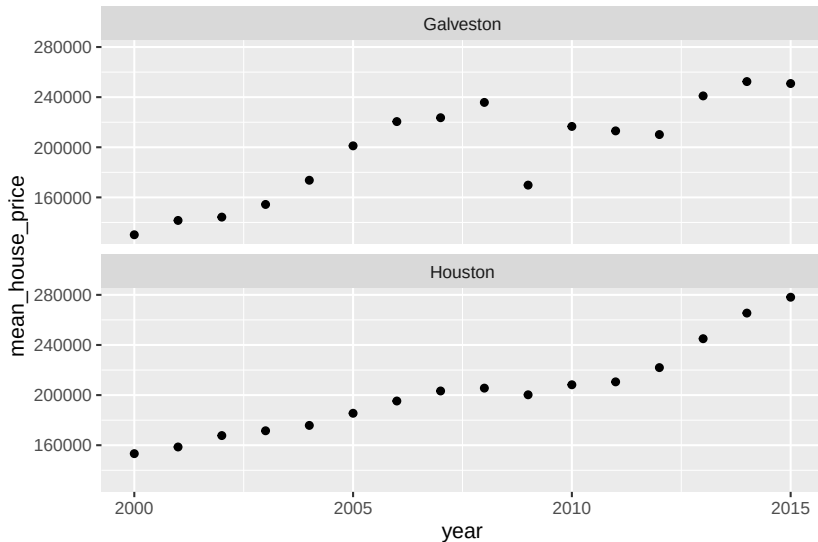
```
annual_city_housing_prices <-  
  housing_data %>%  
    group_by(city, year) %>%  
    summarize(total_sales = sum(sales, na.rm = TRUE),  
              total_volume = sum(volume, na.rm = TRUE),  
              mean_house_price = total_volume / total_sales)  
  
head(annual_city_housing_prices)
```

```
#> # A tibble: 6 x 5  
#> # Groups:   city [1]  
#>   city      year total_sales total_volume mean_house_price  
#>   <chr>   <dbl>         <dbl>         <dbl>         <dbl>  
#> 1 Abilene 2000           1375      108575000      78964.  
#> 2 Abilene 2001           1431      114365000      79920.  
#> 3 Abilene 2002           1516      118675000      78282.  
#> 4 Abilene 2003           1632      135675000      83134.  
#> 5 Abilene 2004           1830      159670000      87251.  
#> 6 Abilene 2005           1977      198855000     100584.
```

How have Texas housing prices changed over time in certain cities?

```
# We will cover data visualization soon!
annual_city_housing_prices %>%
  filter(city %in% c("Houston", "Galveston")) %>%
  ggplot(aes(x = year, y = mean_house_price)) +
    geom_point() +
    facet_wrap(facets = "city")
```

How have Texas housing prices changed over time in certain cities?



Grouping + Summarizing: Base R vs Tidyverse

Base R:

```
aggregate(sales ~ city + year,  
          housing_data,  
          mean)  
aggregate(sales ~ city + year,  
          housing_data,  
          sd)
```

Tidyverse:

```
housing_data %>%  
  group_by(city, year) %>%  
  summarize(mean_sales = mean(sales, na.rm = TRUE),  
            sd_sales = sd(sales, na.rm = TRUE))
```

Recap

We learned:

- ▶ how to employ the dplyr verbs of highest importance including
 - ▶ `rename()` to rename the columns
 - ▶ `mutate()` to create new columns with `ifelse()` and `case_when()`
 - ▶ `summarize()` to summarize data
 - ▶ `group_by()` to make analysis across different subgroups

Appendix

Piping dplyr verbs together

dplyr verbs can be piped together in any order you want, although different orders can give you different results, so be careful!

```
housing_data %>%  
  select(city, year, month, sales, volume) %>%  
  mutate(log_mean_price = log(volume / sales)) %>%  
  filter(year == 2013) %>%  
  summarize(log_mean_price_t = mean(log_mean_price, na.rm = TRUE))
```

► `mean(log_mean_price) = 12.0582`

Piping dplyr verbs together

However, the codes above won't give you the same previous result:

```
housing_data %>%  
  select(city, year, month, sales, volume) %>%  
  mutate(log_mean_price = log(volume / sales)) %>%  
  summarize(log_mean_price_t = mean(log_mean_price, na.rm = TRUE)) %>%  
  filter(year == 2013)
```

- ▶ Actually this code will give you an error message...
- ▶ Try to figure out the reason!